

目次

進化計算による「LLM 駆動 MAP-Elites を安全に走らせる言語」探索—実験報告	1
1. 目的	1
2. 実験構成	2
3. パイプライン実行	3
4. 結果	4
5. 進化計算が示唆する設計原理	6
6. 進化計算が生んだ言語 PhiLang	6
7. 限界と次のステップ	9
8. まとめ	9
参考: 主要ファイル (このディレクトリ内)	10
参考: 親実験	10

進化計算による「LLM 駆動 MAP-Elites を安全に走らせる言語」探索—実験報告

日付: 2026-05-16 実験名: 2026-05-16_robustness_v1 作業ディレクトリ: /Users/kodamay/ocaml-app/abclcp-project/aice-pi-evolution/experiments/2026-05-16_robustness_v1/ 親実験: REPORT_JA.md (PiLang を生んだ 2026-05-15 実験)

1. 目的

PiLang を生んだ進化計算は「 π 10,000 桁を求めるのに最適な言語」を探索した。本実験は同じパイプラインをメタ層に当て、

「LLM 駆動 MAP-Elites を実機上で安全 (コスト $\leq$ \$2 / 壁時計 $\leq$ 30 分 / silent death 0) に完走させる」

という問題を解くのに最適なランタイム構成言語を進化計算で探索する。元の PiLang 探索が「数値計算問題から言語を生む」だったのに対し、本実験は「メタ計算問題から言語を生む」点で対照的である。

具体的には、2026-05-16 セッションの前半で観測した 3 つの失敗モード (試行#1~#3) を踏まえ、.aice (YAML) $\rightarrow$.ga.json $\rightarrow$.aipl のパイプラインを通して、

- 11 軸 (provider, model, concurrency, seed_count, generations, request_timeout_s, sdk_retries, drain_timeout_s, idle_ms, checkpoint_per_gen, caffeinate) を持つランタイム遺伝子を設計、
- F1~F5 の 5 つの reviewer を LLM-as-judge として実装、
- OpenAI gpt-4o-mini で MAP-Elites を 30 世代走らせ、
- 個体群から「安全に走るための設計原理」を抽出、
- それを表現する言語 **PhiLang** を提案する、

までを一気通貫で行った。

2. 実験構成

2.1 ディレクトリ

experiments/2026-05-16_robustness_v1/

└── Pi_Phase1_Robustness.aice	← 案 1 (YAML) の問題定義
└── Pi_Phase1_Robustness.math.aice	← 案 3 (数理最適化) の問題定義 (参考)
└── Pi_Phase1_Robustness.ga.json	← 中間 IR
└── Pi_Phase1_Robustness.aipl	← 実行プログラム (21 KB)
└── robustness_paradigm.schema.json	← 遺伝子スキーマ
└── Pi_Phase1_Robustness.aipl_lineage.json	← 38 個体の結果
└── ai_usage.json	← OpenAI 課金
└── Pi_Phase1_Robustness.log	← 実行ログ
└── REPORT.md	← この文書

2.2 新規遺伝子スキーマ robustness_paradigm.schema.json

PiLang スキーマ (pi_paradigm.schema.json) を継承せず、メタ層用に独立に設計した。11 軸:

軸	値域	由来
provider	openai, anthropic, vertex, mock	LLM 接続先
model	gpt-4o-mini, gpt-4o, claude-haiku-4-5	モデル
concurrency	c1, c2, c4, c8, c16, c32	ABCL_AI_MAX_CONCURRENT
seed_count	s4, s6, s8, s12, s16, s24	MAP-Elites 種数
generations	g10, g15, g20, g30, g40, g60, g80	MAP-Elites 世代数
request_timeout_s	t10, t30, t60, t120, t300	F2 (AIPL_AI_REQUEST_TIMEOUT)
sdk_retries	r0, r1, r3, r5	F2 (AIPL_AI_SDK_RETRIES)
drain_timeout_s	d60, d600, d3600, d7200, d14400	F1 (--timeout)
idle_ms	i1000, i5000, i30000, i60000, i300000	F1 (--idle-ms)
checkpoint_per_gen	off, on	F3 (lineage dump 頻度)
caffeinate	off, on	F5 (Mac sleep 対策)

7 軸を ordinal_axes で順序付け、6 つの coherence ルール (例: provider=openai ⇒ drain_timeout_s ∈ {d3600, d7200, d14400}) で「明らかに不安全な組合せ」を変異が生まないようにした。open_axes=false
— ランタイム設定なので LLM が新軸を提案する余地はない。

2.3 失敗モード F1~F5 を 5 reviewer に展開

2026-05-16 セッション前半で観測した失敗モードを 1 mode = 1 reviewer に対応させた:

reviewer	検出対象	weight	スコア基準
F1_IdleTimeout-Guard	drain_timeout_s と idle_ms が real LLM 用に十分か	0.20	d3600+ & i30000+ → 1.0, 片方 → 0.5, 両方未満 → 0.0
F2_SdkHangGuard	request_timeout_s と sdk_retries で SDK ハング回避	0.25	t30..t120 & r3+ → 1.0, t10..t300 & r1+ → 0.5
F3_Checkpoint-Guard	checkpoint_per_gen=on か	0.15	on → 1.0, off → 0.0 (二値)
F4_Throughput-Guard	30 分以内に完走可能か (total_calls / (23·max(1, α · c)))	0.25	wall ≤ 15min → 1.0, ..., > 60min → 0.0
F5_SleepGuard	wall > 30min なら caffeinate=on か	0.15	推定時間 ≤ 30min → 1.0, > 30min かつ on → 1.0, それ以外 → 0.0

各 persona には **prefix** エンコーディングキー (drain_timeout_s の d{N} は N 秒, d3600=1 時間) と **0.0 – 1.0 段階スコア基準** を明記。これは smoke 縮小版 (4 seed × 10 gen) で reviewer が OK/NG 二値を返そうとして全スコアが 0.04~0.23 に張り付いた問題を直したもの。

2.4 .aice から .aipl までのパイプライン

PiLang 実験で確立した .aice → .ga.json → .aipl の三段パイプラインをそのまま流用。今回は .aice の dialect が **YAML (案 1)** であるため、aice_parser.lower() の curly-brace パーサは使えず、**YAML を手作業で .ga.json にマッピング**した。ga.json → .aipl は既存の aipl_codegen.generate_program() がそのまま動いた (641 行の AIPL プログラムを生成)。

3. パイプライン実行

3.1 試行履歴 (3 失敗 + 1 完走)

試行	結果	教訓
#1	5 秒で死亡 (1 call)	--timeout 2.0 / --idle-ms 120 の既定値が real LLM では短すぎ
#2	gen 38/40 (1267 calls, \$0.44) で silent death	OpenAI SDK の HTTPS request にタイムアウト無で永久待機
#3	gen 11/40 (23 個体, \$0.20) でユーザ中断	実効並列度 1~2 でスループット遅い
smoke	14 個体 (350 calls, \$0.094, 13 分) 完走	reviewer OK/NG 表記で score 低 (max 0.23)
#4 本番	38 個体 (950 calls, \$0.269, 33 分) 完走	reviewer 0.0 – 1.0 + prefix キー化で score 改善 (max 0.46)

3.2 #4 本番ランの安全策

```
AIPL_AI_PROVIDER=openai
ABCL_AI_MAX_CONCURRENT=8
AIPL_AI_REQUEST_TIMEOUT=60      # F2 対策
AIPL_AI_SDK_RETRIES=3           # F2 対策
--timeout 7200                  # F1 対策
--idle-ms 60000                 # F1 対策
```

これら 4 つの env と CLI フラグは試行 #1～#3 から学んだ「失敗を防ぐ既定」である。興味深いことに、進化計算は同じ結論にメタ的に到達した (§4 参照)。

3.3 サマリ

指標	実測	予測	差
完走	✓	—	—
壁時計	33 分	45 分	– 27%
コスト	\$0.269	\$0.34	– 20%
コール数	950	1,026	– 7%
スループット	30 calls/min	23 calls/min	+30%
silent_death	0	0	✓

4. 結果

4.1 個体軸頻度 (38 個体, top 3)

軸	上位値	コメント
provider	openai:14, mock:13, vertex:11	偏り少 (rng_seed=31415926 起因の偶然均一)
model	gpt-4o:15, claude-haiku-4-5:14, gpt-4o-mini:9	reviewer は model ID をテキスト判定するだけなので gpt-4o-mini が必ずしも勝たない
concurrency	c8:9, c16:9, c1:8, c32:7, c2:5	top 5 では c1 が支持 (g15 だと並列度に依存しない)
seed_count	s8:13, s6:8, s12:8, s4:5, s16:4	中央値が好まれた
generations	g15:14 , g10:6, g20:5, g30:4, g40:4	「短いほど完走確率高い」が圧倒的多数派
request_timeout_s	t60:11, t30:10, t120:8, t300:5, t10:4	F2 推奨域 t30–t120 に集中
sdk_retries	r3:12, r5:11, r1:8, r0:7	r3+ が 23/38 (60%) で F2 推奨を満たす
drain_timeout_s	d7200:11, d14400:8, d3600:7, d600:7, d60:5	d3600+ が 26/38 (68%) で F1 推奨を満たす

軸	上位値	コメント
idle_ms	i60000:11, i30000:10, i300000:8, i5000:5, i1000:4	i30000+ が 29/38 (76%) で F1 推奨を満たす
checkpoint_per_gen	off:22, on:16	F3 weight=0.15 が他に押し負け
caffeinate	off:20, on:18	ほぼ五分

→ F1 と F2 の reviewer が強い選択圧を生み、F3 は weight 不足で機能不全、という非対称な学習結果。

4.2 Top 5 個体の遺伝子

順位	id	gen	op	score	genome (要約)
1	l25	17	axis_resam- ple	0.457	mock-gpt-4o · c1·s6·g15 · t30·r3·d14400·i30000 · ckpt=off·caff=off
2	l24	16	uni- form_crossover	0.455	mock-gpt-4o · c8·s8·g15 · t300·r5·d600·i30000 · ckpt=off·caff=on
3	l19	11	uni- form_crossover	0.421	ope- nai-claude- haiku · c2·s8·g15 · t30·r5·d7200·i60000 · ckpt=off·caff=off
4	l1	0	seed	0.397	mock-gpt-4o · c1·s6·g15 · t30·r3·d600·i30000 · ckpt=off·caff=off
5	l30	22	axis_resam- ple	0.396	ope- nai-claude- haiku · c32·s8·g15 · t30·r5·d7200·i60000 · ckpt=off·caff=off

4.3 世代別スコア推移

```
gen= 0: max=0.397 mean=0.257    (8 seed の初期スコア帯)
gen= 4: max=0.392
gen=11: max=0.421                (crossover 1 発目で上振れ)
gen=16: max=0.455
gen=17: max=0.457                ← 全体ベスト
gen=22: max=0.396
```

→ 25 世代で 0.397 → 0.457 に 約 15% 改善。改善幅は小さいがブレイクスルー (gen 16-17) は crossover が生み出した。MAP-Elites の elite 更新と突然変異の組合せが機能していることが分かる。

5. 進化計算が示唆する設計原理

38 個体のスコア分布と top 5 の共通項から、「LLM 駆動 MAP-Elites を安全に走らせる」言語が**先天的に備えるべき四つの特性**を抽出する。これは PiLang における Array_DSL × refinement-typed × region-owned × streaming の 4 つ巴と同じ抽象度の知見である。

特性	進化計算が示した根拠
(A) Bounded burst	top 5 全員が generations=g15 を支持。長時間ランより短バースト + 部分結果採用を選好
(B) Mandatory retry/timeout	全 top 5 が sdk_retries ≥ r3 かつ request_timeout_s ≤ t300。「待ち続ける」より「諦めて再試行」
(C) Generous drain buffer	80% が drain_timeout_s ≥ d3600 & idle_ms ≥ i30000。actor 落穂拾いに余裕を持つ
(D) Forced checkpoint	進化計算自体は F3 を見落としたが、人間が安全運用のために 必須化すべき特性 (§7.4 参照)

→ 「**Bounded burst × Mandatory retry × Generous drain × Forced checkpoint**」の交点として、メタ問題に最適な言語が浮かび上がる。

6. 進化計算が生んだ言語 PhiLang

PiLang が π の 1 タスクから生まれた言語であるように、本実験は**メタ・実行制御**の 1 タスクから **PhiLang** (Φ = Phase, 「位相」) を生んだ。

6.1 PhiLang のコア構文

```
# PhiLang — Phase Orchestration Language synthesized from MAP-Elites
# (2026-05-16_robustness_v1 evolution result)
```

```
phase Phase1
  within 30 min      and $0.30                # ハード予算
  uses   openai/gpt-4o-mini
```

```

drains    for 2 h      idle 60 s          # actor drain buffer
caffeine  if duration > 30 min           # F5
{
  seeds      = 8
  generations = 15                # ← 短バースト (A)
  cell_axes  = [paradigm, arithmetic, algorithm, precision, memory]

  every call has policy {          # ← 全コールに強制
    timeout    : 60 s
    retries    : 3 with exponential_backoff    # ← (B)
    on_hang    : kill_and_retry
  }

  after every individual {         # ← 強制 checkpoint (D)
    checkpoint lineage to "out/lineage.json"
  }
}
guarantees {
  no_silent_death                # ← (B), (C) から導出
  bounded_cost                   $0.30
  bounded_wall                   30 min
  partial_resume_on_kill         # ← (D) から導出
}

```

3 ブロック (phase / every call has policy / after every individual) + 1 契約ブロック (guarantees) で書ける。every call has policy は言語の意味論レベルでレキシカルスコープ内の全 ai_call に強制適用される (オプティンではない)。

6.2 脱糖形 (AIPL ランタイム呼び出し)

PhiLang の上記 1 ブロックは現状の AIPL ランタイムに対しては次のように脱糖される:

```

# 環境変数 (every call has policy → SDK 設定)
ABCL_AI_MAX_CONCURRENT      = 8          # phase 内既定
AIPL_AI_REQUEST_TIMEOUT     = 60         # policy.timeout
AIPL_AI_SDK_RETRIES         = 3          # policy.retries
AIPL_AI_TOKEN_BUDGET        = 3_000_000 # within $0.30 から逆算

# CLI フラグ (drains 句)
--timeout 7200              # drains for 2 h
--idle-ms 60000             # idle 60 s

# AIPL コード (after every individual)
while (gen <= 15) do {
  ...                      # 個体生成と評価
  send lineage.add(...)
  send elite.propose(...)
}

```

```

    var n = now lineage.dump("out/lineage.json");    # ← 強制 checkpoint
    gen = gen + 1;
}

```

```

# シェルラップ (caffeine 句)
caffeinate -i &                # PID をフェーズ終了で kill

```

6.3 4 つの言語要素が解決する失敗モード

PhiLang の構文要素	解決する失敗モード	進化計算での根拠
within 30 min and \$0.30 (ハード予算)	F4 (low_throughput)	top 5 全員が g15 → 短バースト原則
every call has policy { timeout/retries }	F2 (sdk_hang)	top 5 全員が r3+ で SDK 強制リトライ
drains for 2h idle 60s after every individual { checkpoint }	F1 (idle_timeout_too_short) F3 (no_checkpoint)	80% が d3600+, i30000+ 進化計算が見落としたので言語が必須化
caffeine if duration > 30min	F5 (macos_sleep)	reviewer F5 が学習済の判定式
guarantees { ... } 節	全 F1～F5 を契約として明文化	個体群から導出した invariants

6.4 PhiLang の意味論的特徴

■6.4.1 「コールサイト無し」のリトライ/タイムアウト PhiLang では `ai_call(...)` 一文一文にリトライやタイムアウトを書かない。 `every call has policy` ブロックがレキシカルに効くため、生 LLM API のサブセットは存在しない。これは試行 #2 で SDK が暗黙にハングした問題を「言語の表現可能性」レベルで除去する。

■6.4.2 「チェックポイント無し phase は型エラー」 PhiLang のフェーズは `after every individual { checkpoint }` 句が必須。これを書き忘れると型検査で `non-resumable phase: missing checkpoint clause` を出す。試行 #2 で 38/40 個体分のスコアが `silent death` で消えた問題をコンパイル時に防ぐ。

■6.4.3 `guarantees` 節は実行可能 invariant `bounded_cost $0.30` は単なるコメントではなく、ランタイムが `ai_usage.json` を監視して超過時に `BudgetExceeded` を投げる。`no_silent_death` は `every call has policy` の存在から自動証明される。型と契約が双方向に検証される。

■6.4.4 「メタ進化」のためのリフレクション PhiLang は自身のフェーズを進化計算の対象にできる。本実験の .aice (YAML) に書かれた 11 変数は、PhiLang で書けば `evolvable` 修飾子付きパラメータになる：

```

phase Pi_Phase1_Robustness
  evolvable concurrency in [c1, c2, c4, c8, c16, c32]
  evolvable seed_count in [s4, s6, s8, s12, s16, s24]
  evolvable generations in [g10, g15, g20, g30, g40, g60, g80]
  ...
  optimize for {
    maximize completed_generations

```



```

    minimize cost_usd
}

```

→ **PhiLang** は **MAP-Elites** の対象であると同時に、**MAP-Elites** を記述する言語でもある。これは **PiLang** が自身を進化させないのに対し、**PhiLang** 固有の自己参照性である。

7. 限界と次のステップ

7.1 reviewer が見落としした特性 (F3 と provider 不整合)

§4.1 の通り、`checkpoint_per_gen=off` が top 5 で 5/5、`provider=mock` が top 5 で 3/5 を占めた。これは **reviewer** の設計バグであり **PhiLang** の知見ではない。次イテレーション (v2) で以下を直す:

1. **F3** を **hard floor** 化: `checkpoint_per_gen=off` の個体は最終スコアを $\times 0$
2. **mock provider** を最終スコア化前に除外
3. **provider $\times$ model coherence rule** 追加: `openai` $\Rightarrow$ `model` $\in$ `{gpt-4o, gpt-4o-mini}` 等

7.2 PhiLang のコンパイラが未実装

§6 の **PhiLang** は設計段階に留まっている。実装するには:

1. **PhiLang** フロントエンド (`phi_parser.py`) を書く。`phase / every / after / guarantees / evolvable` の 5 ブロックを AST に。
2. `phi_codegen.py` で env 設定 + CLI フラグ + AIPL コードに脱糖。
3. `guarantees` 節を `aipl_typeck.py` の追加チェックとして実装。

工数概算: 1~2 日。

7.3 YAML lowerer の欠落

`aice_parser.lower()` が curly-brace `.aice` 用なので、YAML 形式の本実験 `.aice` は手作業で `.ga.json` にマップした。再現性のためには `aice_parser_yaml.py` を新規に書くか、既存 `lowerer` に `dialect` 検出を追加する必要がある。

7.4 進化軸が現実の API 制約を知らない

`provider  $\times$  model` の組合せ (`openai` に `claude-haiku`) のような実在しない組合せを **reviewer** は弾けなかった。これは **reviewer** の知識の限界。次は **coherence rule** を充実させるか、**reviewer** の **persona** に「実在する `provider $\times$ model` 対の白リスト」を埋め込む。

8. まとめ

項目	結果
案 1 (YAML) <code>.aice</code> 設計	11 軸 / 5 reviewer / 5 task / 4 plan branch を 230 行で記述
案 3 (Math) <code>.aice</code> 設計 (参考)	同じ問題を <code>decision variables + objective + constraints</code> で記述 (270 行)
<code>.ga.json</code> 生成	手作業マッピング (<code>codegen</code> 互換)

項目	結果
.aipl 生成	21 KB / 641 行、parse OK、smoke + 本番ともに完走
AIPL 本番ラン	33 分 / \$0.269 / 950 calls / 38 個体 / silent_death=0
進化計算が選んだ言語	PhiLang —Bounded burst × Mandatory retry × Generous drain × Forced checkpoint を言語の意味論で強制
副次成果	reviewer 設計の弱点 (weight 不足 / 不在 coherence) を 3 点同定

本実験は「メタ計算問題からプログラミング言語を進化計算で生み出す」という仮説を、PiLang (π 数値計算問題) と同じ .aice → .ga.json → .aipl パイプライン上で端から端まで動作させた第 2 の実証である。

PiLang が「特定の数値計算を最短に書く言語」だったのに対し、PhiLang は「自分自身の進化計算実行を最も安全に書く言語」である点で対照的だが、両者とも

- ・ MAP-Elites の個体群から共通する設計原理を抽出し、
- ・ その原理を意味論レベルで強制する 3~4 行の最小構文に結晶化する、

という同じ方法論を踏襲している。aice-evolution-v2 パイプラインは問題ドメインに依存しない普遍的な「言語設計支援ツール」として機能することが、今回 PiLang 以外のドメイン (メタ層) で初めて検証された。

参考: 主要ファイル (このディレクトリ内)

- ・ 問題定義: Pi_Phase1_Robustness.aice (YAML), Pi_Phase1_Robustness.math.aice (数理参考)
- ・ スキーマ: robustness_paradigm.schema.json
- ・ 中間 IR: Pi_Phase1_Robustness.ga.json
- ・ AIPL プログラム: Pi_Phase1_Robustness.aipl
- ・ 実行結果 lineage: Pi_Phase1_Robustness.aipl_lineage.json
- ・ 課金記録: ai_usage.json
- ・ 実行ログ: Pi_Phase1_Robustness.log

参考: 親実験

- ・ aice-pi-evolution/REPORT_JA.md — 2026-05-15 の PiLang 探索報告。本実験はその「同じ方法論を別ドメインに適用した第 2 例」に相当する。